

CS 211

Project 4 – Reverse Polish Expression Evaluator

1. Introduction

In this project, you will build a simple interactive calculator.

When you have completed the project, interaction with `rpncalc.py` will look like this:

```
Expression (return to quit):3 5 * x =  
x = (3 * 5) => 15  
Expression (return to quit):x 2 *  
(x * 2) => 30  
Expression (return to quit):  
Bye! Thanks for the math!
```

1.1. Objectives

This programming project allows you to practice the following:

- *Refactoring* redundant code from concrete subclasses into a shared abstract base class.
- Using dynamic dispatch (polymorphism) to tie together refactored code.
- Recursive traversal of object structures. `__str__`, `__repr__`, and `eval` will make recursive calls in a style that is a little different from the typical recursive functions you have written before.

1.2. Outcomes

After doing this project, you will understand the concepts of

- Refactoring in Python
- Class inheritance
- Method overriding
- Dynamic dispatch as a form of polymorphism

2. Project activities

In this project, you will create a calculator, building the core functionality (module `expr.py` with classes representing arithmetic expressions) and the calculator that uses it (module `rpncalc.py`).

The calculator uses reverse Polish notation (RPN), also called *postfix notation*, in which we write the expression `(5 + 3) * 4` as `5 3 + 4 *`. You will write code to translate this postfix expression into a tree.

A Python module, `expr.py`, will define the classes for expression tree nodes. The calculator will use those classes implemented in the module `rpncalc.py`.

You will develop a recursive data structure (objects) to represent expressions. Based on that data structure, you will implement an algorithm (with its related classes) to transform postfix expressions to objects of the `Expression` class.

2.1. Expressions

You will represent expressions as objects. The expressions you will include are:

- Numeric constants, e.g., `5`. The value of an integer constant is its numerical value. An object of class `Const` represents an integer constant. A call to the constructor has the form `Const(5)`.
- Variables, e.g., `x`. Our calculator will allow us to assign values to variables. The value of a variable is whatever value it is currently holding. (More on this below.)
- Operators.
 - Binary operators (e.g., `+` and `*`). We call them *binary* because they take two operands. For example, we might represent `5 + 3` as `Plus(Const(5), Const(3))`. To determine the value of a binary operation, apply that operation to the values of its operands. For example, `Plus(Const(5), Const(3)).eval()` would return `Const(8)`.
 - Unary operations (e.g., number negation and absolute value), in contrast, have one operand only. For example, we might represent `Minus(3)` (written as `~3` in algebraic notation) as `Minus(Const(3))`. In algebraic notation, we write “|3|” to denote the absolute value of 3. In this project we’ll use `@` as the absolute value operator; the expression “3 @” should produce `Abs(Const(3))`.
 - Assignment. When we add variables, we want to add an operation for assigning a value to a variable, e.g., `x = 3` would be `Assign(Var('x'), Const(3))`.
- Define a single *abstract* class `Expr` and make all kinds of expressions (e.g., `Const`, `Plus`, etc.) subclasses of `Expr`. All subclasses of `Expr` should implement an `eval` method that returns the value obtained by evaluating the expression. For example, the expression `Times(Plus(Const(5), Const(3)), Const(4)).eval()` returns `Const(32)`.

In addition to `eval`, each kind of expression must have a `__str__` method to return its string representation in algebraic notation and `__repr__` to return a string that looks like a call to construct the expression. For example, the `str` representation of

`Times(Plus(Const(5), Const(3)), Const(4))` should return `"(5+3)*4"`, and its `repr` form should return `"Times(Plus(Const(5), Const(3)), Const(4))"`. Getting minimal parentheses in the `__str__` method is slightly complicated, so our `__str__` method will over-parenthesize expressions with binary operations. Unary operations do not present the same ambiguity issues, so we will not need additional parentheses for negation and absolute value.

Class Expr

Create the file `expr.py`, which contains all the classes for representing expression trees (include a file docstring).

We start with the abstract base class `Expr`, which serves as a template and documentation for concrete subclasses; each should implement `eval`, `__str__`, and `__repr__`.

```
class Expr():
    """Abstract base class of all expressions."""

    def eval(self) -> "Const ":
        """Implementations of eval should return an integer constant."""
        raise NotImplementedError(
            f'eval' not implemented in {self.__class__.__name__}\n"
            "Each concrete Expr class must define 'eval'"
        )

    def __str__(self) -> str:
        """Implementations of __str__ should return the expression
        in algebraic notation"""
        raise NotImplementedError(
            f'__str__' not implemented in {self.__class__.__name__}\n"
            "Each concrete Expr class must define '__str__'"
        )

    def __repr__(self) -> str:
        """Implementations of __repr__ should return a string that
        looks like the constructor,
        e.g., Plus(Const(5), Const(4))
        """
        raise NotImplementedError(
            f'__repr__' not implemented in {self.__class__.__name__}\n"
            "Each concrete Expr class must define '__repr__'"
        )
```

The error messages these abstract methods produce use dunder attributes. `t.__class__` obtains the class of object `t`, and `__name__` gets the name of that class as a string. Note that the error message spans two lines. Note that Python interprets the juxtaposition of two string literals as concatenation, e.g., `"a" "b"` produces `"ab"`.

Checkpoint

At this point, you have `expr.py` with one abstract class, `Expr`. You can test this by starting a Python console in the terminal. Import the `Expr` class from `expr.py`, create an `Expr` object, and attempt to print it.

```
$python
Python 3.11.5 (main, Sep 11 2023, 08:31:25) [Clang 14.0.6 ] on Darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> from expr import Expr
>>> e = Expr()
>>> print(e)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/Users/michal/Dropbox/23W-211/projects/dist/calculator/expr.py", line 14, in __str__
    raise NotImplementedError(
NotImplementedError: '__str__' not implemented in Expr
Each concrete Expr class must define '__str__'
```

Const

The first concrete subclass we will build is **Const** because the **eval** method returns an object of class **Const**; we cannot test anything else without it.

Next, we write the actual class, filling in the needed methods in the usual way. You will need to create class **Const** and methods **__init__**, **eval**, **__str__**, and **__repr__**. A **Const** object should have one instance variable, a **number**. It should have the same **string** representation as the **numeric** value it wraps. Its **repr** should look like a call to the **Const** constructor. A constant evaluates to itself, i.e., the **eval** method can just **return self**.

To compare objects of the same class, we need to add an **__eq__** method to **Const**. Note that we need to be able to compare any **Expr** object to any other one. The expression **Const(7) == Plus(Const(3), Const(4))** should return **False** (it is not the same tree, even if it would evaluate to the same value). Still, it would instead raise an exception because a **Plus** object does not have a **value** field (instance variable). See the subsection **Plus** below.

Python provides the **isinstance** function to check whether an object belongs to some class. We will use it so that we only access the instance variables of **other** if it is a constant. Objects of different types will never be considered equal:

```
def __eq__(self, other: Expr):
    return isinstance(other, Const ) and self.value == other.value
```

Checkpoint

Currently, your **expr.py** file contains the abstract class **Expr** and the concrete subclass **Const**.

Plus

Start with one binary operation, **Plus** (represented by **+** in the input). We detected something tricky here before we started writing code for the **Plus** class. How will we get parentheses around the nested sub-expression **(5 + 4)** but not around the whole thing? Maybe for **Plus**,

we could just omit all the parentheses, but we could not do that for **Minus** because $(3 - 4) - 2$ is not the same as $3 - (4 - 2)$. We will over-parenthesize.

Given that, write the **class Plus**, starting with the constructor. It should store a left operand as **self.left** and a right operand as **self.right**.

__str__ is recursive

The **__str__** method of **Plus** makes implicit recursive calls on the **__str__** methods of the **left** and **right** operands:

```
def __str__(self) -> str:
    """Algebraic notation, fully parenthesized: (left + right)"""
    return f"({self.left} + {self.right})"
```

How does this recursion work? (If you haven't already, now would be a good time to read the chapter on recursion in object-oriented programs – https://uo-cs-oer.github.io/CS211-text/03_Recursion/03_1_Recursion.html.) We must determine whether to apply the base case or the recursive case in recursive functions. In this method, the base and recursive cases are not combined in a single function body; instead, they are in separate subclasses. We get the base case if we call **str** on a **Const** object. We get the recursive case if we call **str** on a **Plus** object. The **__repr__** method works like the **__str__** method – write it.

Checkpoint

Execute the following expressions – your implementation so far should produce similar results.

```
>>> a = Const(7)
>>> b = Const(5)
>>> res = Plus(a, b)
>>> print("a =", a)
a = 7
>>> print("b =", b)
b = 5
>>> print("res =", res)
res = (7 + 5)
>>> print([res])
[Plus(Const(7), Const(5))]
```

Expressions are trees

You can visualize an expression as a tree. In software development and computer science, we typically draw trees upside down, with the *root* (*root node*) at the top and *the leaves* (*leaf nodes*) at the bottom.

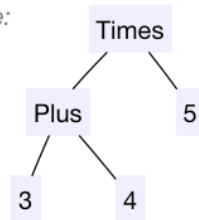
Algebraic notation: $(3 + 4) * 5$

Reverse Polish notation: $3\ 4\ +\ 5\ *$

Object literal (constructor calls):

`Times(Plus(IntConst(3), IntConst(4)), IntConst(5))`

Tree:



Viewed this way, the base case of the recursion is in the methods in the leaf nodes, and all other nodes perform the recursive case. You can view the recursive computation as *walking* the tree, systematically working down to the leaves, and returning values upwards.

Recursive eval

The `__str__` and `__repr__` methods made recursive calls using the `left` and `right` operands. So does `eval` for the plus operator, which we can code as:

```
def eval(self) -> "Const ":
    """Implementations of eval should return an integer constant."""
    left_val = self.left.eval()
    right_val = self.right.eval()
    return Const(left_val.value + right_val.value)
```

Checkpoint

You now have the base class `Expr` and two concrete subclasses, `Const` and `Plus`, enough to represent expressions that involve only addition, like `Plus(Plus(Const(5), Const(3)), Const(4))`. You can check the functionality in the Python console:

```
>>> from expr import *
>>> Plus(Plus(Const(5), Const(3)), Const(4)).eval()
Const(12)
```

2.2. Reverse Polish Notation

Reverse Polish notation (RPN – aka reverse Łukasiewicz notation or postfix notation) is a mathematical notation in which operators *follow* their operands. The name *Polish* refers to the nationality of logician Jan Łukasiewicz, who invented it in 1924.

In RPN, instead of $3 + 4$, we write $3\ 4\ +$; we can write $(3*4)/6$ as $3\ 4\ *\ 6\ /$, and $3*(4/6)$ as $3\ 4\ 6\ /\ *$. Notice that RPNs do not need parentheses!

The goal is to build an RPN expression parser and evaluator; the parser uses a **stack**. An operand is *pushed* onto the **stack**. A binary operator pops two elements from the **stack** and pushes the result back onto the **stack**. In Python, a list can be used as a **stack**, using the **append** and **pop** list methods.

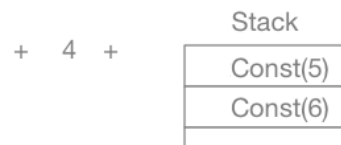
Let us consider the parsing sequence for **6 5 + 4 +**. Initially, the **stack** is empty.



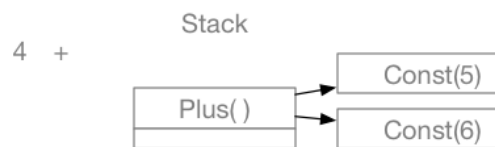
We process the expression from left to right. Find the first token (**6**), create a corresponding **Const** object, and push it onto the **stack**.



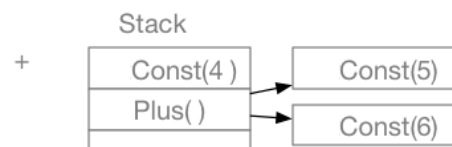
Do the same for the next value.



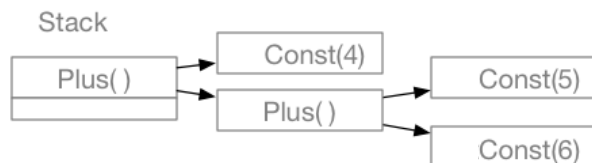
The next token is **+**. The algorithm creates a **Plus** node for it. Note that the **left** and **right** operands are on the stack. We **pop** them, build the **Plus** object, and **push** it onto the **stack**.



Here, **Plus** is the tree root (we will draw it sideways to draw the **stack** vertically. Next, we have another integer, **4**. We **push** it onto the **stack** just as we did with the others.



Finally, we have another **+**. We treat it like the first: **pop** the two operands, build the **Plus** object, and **push** it. The only difference is that the left operand will be another **Plus** node this time.



After this, if the input was syntactically correct, the top element of the **stack** holds the **Expr** object that represents the whole expression.

Checkpoint

At this point, the only **Expr** classes we have are **Const** and **Plus**, so we cannot build very interesting expressions, but it's nice to run **rpncalc.py** and see that it is working as intended so far:

```
$ python3 rpncalc.py
Expression (return to quit):6 5 4 + +
(6 + (5 + 4)) => 15
Expression (return to quit):6 5 + 4 +
((6 + 5) + 4) => 15
Expression (return to quit):
```

2.3. Back to Expr

So far, we have a way of building and evaluating Expr objects, but our only operation is **Plus**. Let us add **Times**, which might look like this:

```
class Times(Expr):
    """left * right"""

    def __init__(self, left: Expr, right: Expr):
        self.left = left
        self.right = right

    def eval(self) -> "Const ":
        """Implementations of eval should return an integer constant."""
        left_val = self.left.eval()
        right_val = self.right.eval()
        return Const(left_val.value * right_val.value)

    def __str__(self) -> str:
        """Implementations of __str__ should return the expression in
        algebraic notation"""
        return f"({str(self.left)} * {str(self.right)})"

    def __repr__(self) -> str:
        """Implementations of __repr__ should return a string that looks
        like the constructor, e.g., Plus(Const(5), Const(4))
        """
        return f"Times({repr(self.left)}, {repr(self.right)})"
```

2.4. DRY it out!

The code works; you just copied code from the **Plus** class and made a few changes. But repeating almost identical code is not a good idea. The code is error-prone, particularly when we change some of it but fail to update it everywhere else consistently.

Don't repeat yourself (DRY) is a software development principle that encourages developers to avoid repeating code by replacing it with abstractions or using data normalization to avoid redundancy.

Factor out what is common between **Plus** and **Times** (and other arithmetic operators we have not written yet). We would like to create an abstract superclass that contains the common code and write only the code specific to each operator in the class for that operator.

Start by creating a class **BinOp** (for arithmetic operations with two operands) and determine what we can *factor* into it.

```
class BinOp(Expr):
    pass

class Plus(BinOp):
    """left + right"""
    # ... as before

class Times(BinOp):
    """left * right"""
    # ... as before
```

The **BinOp** class does not implement all the methods of **Expr**. That is ok because **BinOp** is an abstract class.

Let's start factoring out the `__str__` method and analyze the differences between the implementations of the classes **Times** and **Plus** we have so far.

```
class Times(Expr):
    """left * right"""
```

```
def __init__(self, left: Expr, right: Expr):
    self.left = left
    self.right = right
```

```
def __str__(self) -> str:
    return f"({str(self.left)} * {str(self.right)})"
```

```
def __repr__(self) -> str:
    return f"Times({repr(self.left)}, {repr(self.right)})"
```

```
class Plus(Expr):
    """left + right"""
```

```
def __init__(self, left: Expr, right: Expr):
    self.left = left
    self.right = right
```

```
def __str__(self) -> str:
    return f"({str(self.left)} + {str(self.right)})"
```

```
def __repr__(self) -> str:
    return f"Plus({repr(self.left)}, {repr(self.right)})"
```

Different

Same

The constructor is the same (setting the **left** and **right** instance variables), and the `__str__` and `__repr__` methods differ only in the operation symbol (+ or *) and the class name ("Plus" or "Times"). We could factor all this common code into the **BinOp** class if we just put the operation symbol and class name in the variables.

```
class BinOp(Expr):
    """Abstract base class for binary operations"""
    def __init__(self, left: Expr, right: Expr, \
                 symbol: str="?Operation symbol undefined"):
        self.left = left
        self.right = right
        self.symbol = symbol
```

Now, the constructors of concrete subclasses can delegate to the constructor of **BinOp**, providing the appropriate symbol:

```
class Plus(BinOp):
    """Expr + Expr"""
    def __init__(self, left: Expr, right: Expr):
        super().__init__(left, right, symbol="+")
```

We have saved a few lines of repetitive code. A bigger gain comes from factoring the `__str__` and `__repr__` methods into `BinOp` and removing them from `Plus`. The `__str__` method in `BinOp` can be:

```
def __str__(self) -> str:
    return f"({self.left} {self.symbol} {self.right})"
```

We can now remove the `__str__` method from the `Plus` class.

Factoring the `__repr__` method is slightly more complex. Recall how we obtained the name of a class in abstract methods in class `Expr` as `self.__class__.__name__`. We can use that to build a generic `__repr__` method in class `BinOp`:

Similarly, we will factor the `__repr__` methods and do the same for all binary operators.

Checkpoint

We have not added any new functionality to this step, but we have simplified the code. Now you have classes `Expr`, `BinOp`, `Plus`, and `Times`. The only methods in `Plus` and `Times` are the constructor (`__init__`) and `eval`. The constructor calls its superclass with one extra argument for the operation `symbol`.

Factoring eval

The only difference between the `eval` methods of `Plus` and `Times` is that the former adds the `left` and `right` values, and the latter multiplies them. We can add an internal method, `_apply`, to each `BinOp` class.

We will need an abstract method `_apply` in the `BinOp` class:

```
def _apply(self, left_val: int, right_val: int) -> int:
    """Each concrete BinOp subclass provides the appropriate method"""
    raise NotImplementedError(
        f"'_apply' not implemented in {self.__class__.__name__}\n"
        "Each concrete BinOp class must define '_apply'")
```

Then the concrete subclasses like `Plus` can override it, for example

```
def _apply(self, left: int, right: int) -> int:
    return left + right
```

This refactoring is more straightforward than the one `eval` requires, which we can now factor into the `BinOp` class like this:

```
def eval(self) -> "Const ":
    """Each concrete subclass must define _apply(int, int)->int"""
    left_val = self.left.eval()
    right_val = self.right.eval()
    return Const(self._apply(left_val.value, right_val.value))
```

At this point, our **Plus** and **Times** classes are short and clear. Let us proceed to write the **Div** and **Minus** classes. Since we are dealing with integers only, **Div** must implement Python's integer truncating division, `//` rather than `/`.

Checkpoint

You should now have **Expr**, **Binop**, **Plus**, **Minus**, **Times**, and **Div** classes. **Plus**, **Minus**, **Times**, and **Div** classes are very similar and short. At this point, they should pass their respective unit test cases.

2.5. Unary operators

Negation, which changes **42** to **-42** and vice versa, is a unary operator. We might represent it in RPN as **~** to avoid ambiguity. The absolute value function is also a unary operation. We customarily write the absolute value of x as $|x|$, but it will be better to have a single symbol – we will use **@**.

Add a new abstract base class, **UnOp**, for unary operations, and add classes **Abs** and **Neg** to implement them. Follow the same general approach as with **BinOp**, except that **UnOp** expressions have only a **left** operand.

Checkpoint

In addition to the **Expr** and all the **BinOp** classes, you have **UnOp** and its concrete subclasses **Neg** and **Abs**.

2.6. Adding variables

A calculator with memory allows the user to manipulate variables, assign values or results of expressions to them, and then use their values in subsequent expressions. Sticking with RPN, we could write.

```
24 60 * 60 * seconds_per_day =
30 seconds_per_day *
```

We will store variables in an environment object – a Python **dict**. Make the environment a global variable in the **expr** module; near the beginning of **expr.py**, we can add:

```
# One global environment variable
from typing import Dict
ENV: Dict[str, "Const "] = {}
```

Use all-caps for **ENV** to remind readers that it is a global variable. The type annotation must place **Const** in quotes because the **Const** class is defined later in the file.

We will create a variable class – evaluating a variable looks up its value in the environment. When assigning a value to a variable, the program stores a value in the environment. The **Var** class should look like this.

```
class Var(Expr):

    def __init__(self, name: str):
        self.name = name

    def __str__(self):
        return self.name

    def __repr__(self):
        return f"Var({self.name})"

    def eval(self):
        global ENV
        if self.name in ENV:
            return ENV[self.name]
        else:
            raise UndefinedVariable(f"{self.name} has not been \
                                   assigned a value")
```

A variable has a name, and **ENV** stores a (variable, value) association. If a variable exists but **ENV** does not contain a value for its name, the eval method raises an exception. To define a new exception, we create a class that inherits from **Exception** (we do not need to provide any additional functionality).

```
class UndefinedVariable(Exception):
    """Raised when expression tries to use a variable that
    is not in ENV
    """
    pass
```

Now we can make an **Expr** subclass **Var**. Its **eval** method will look in the environment for a value that should be a **Const**.

You must provide a method in the **Var** class to assign a value to a variable.

```
def assign(self, value: Const ):
    # You fill this in
    pass
```

Implement that method.

3 5 + x = is an example of an assignment expression that assigns the value **8** to the variable **x**.

The **eval** method in other **Expr** classes returns an **Const** . For consistency, we will code an **Assign** class with an **eval** that sets and returns a variable value.

The **left** operand of an **Assign** object must always be a **Var**:

```
class Assign(Expr):
    """Assignment:  x = E represented as Assign(x, E)"""

    def __init__(self, left: Var, right: Expr):
        assert isinstance(left, Var)  # Can only assign to variables!
        self.left = left
        self.right = right
```

Note that the **Assign** class is different from **Var**'s **assign** method. The **Assign** class will use **Var.assign** in its **eval** method. The **eval** method of **Assign** evaluates its right side but not its left side:

```
def eval(self) -> Const :
    r_val = self.right.eval()
    self.left.assign(r_val)
    return r_val
```

Write the **__str__** and **__repr__** for this class.

2.7. rpncalc.py

To parse assignments, you need to add cases to **rpncalc.py** for variables and assignments.

Variables are straightforward; you must provide code for that case.

For assignments, in infix notation, the variable is on the left side of the assignment operator and an expression on its right side, e.g., **x = 3 + 4**. However, in RPN, it is difficult to read **x 3 4 + 5 * 7 / =** (a long expression separates the variable from the **=** symbol.) Writing **3 4 + 5 * 7 / x =** is clearer, so swap **left** and **right**. You can do that by swapping **left** and **right** in the constructor call:

```
right = stack.pop()
left = stack.pop()
# Reverse left and right
stack.append( ... )
```

You must program the **rpn_parse** function; to do that, you need to write the following auxiliary functions:

```
def is_binop(op:str) -> bool:
    pass

def is_unop(op:str) -> bool:
    pass

def is_var(op:str) -> bool:
    pass

def binop_class(op:str) -> "BinOp":
    pass

def unop_class(op:str) -> "UnOp":
    pass
```

The first function answers the question: Is `op` a binary operator? Likewise, the second and third functions determine whether `op` is a unary operator or a variable.

Given an operator `op` (a string), the function `binop_class` returns the class identifier we need to use to construct an object of that class. For instance `binop_class("+")` returns `Plus`. With this in place, you can now perform calculations involving variables:

```
Expression (return to quit):300 speed =
(speed = 300) => 300
Expression (return to quit):10 time =
(time = 10) => 10
Expression (return to quit):speed time * distance =
(distance = (speed * time)) => 3000
```

3. Rubric

The following list indicates the required components and their associated mark points.

- [10 pts] Concrete class Const
- [10 pts] Abstract class BinOp
- [10 pts] Concrete class Plus (refactored)
- [10 pts] Concrete class Times (refactored)
- [10 pts] function `rpn_calc`
- [5 pts] Abstract class UnOp
- [10 pts] Concrete class Neg
- [10 pts] Concrete class Abs
- [5 pts] Method assign of class Var class Neg
- [10 pts] Concrete class Assign
- [10 pts] Auxiliary functions

What to submit

Upload to Canvas.

- `expr.py`
- `rpncalc.py`.

4. Do not Risk Cheating

The academic integrity rules for CS 211 are very strict, and we take them very seriously. Suspected violations are reported to the appropriate UO office, and the penalty for plagiarism includes failing the course. If you consulted with anyone other than the instructional staff of CS 211 (the instructor, graduate teaching assistants, and undergraduate learning assistants) to complete the program, you must explicitly credit them in the docstring comment. If you referred

to any example code aside from the provided documentation and the Python library documentation, you must explicitly cite it. I hope to see many credits in the projects you turn in because you learn more and better when you collaborate in teaching and learning from each other.

This term, you can work in teams of two people – no more than two. Examples of dishonest behavior are:

- Sharing code between teams
- Forming teams of 3 or more people
- Getting code from online sources, e.g., GitHub, YouTube, chatGPT, StackOverflow, etc., and presenting it as your own

You are encouraged to perform internet searches and take one or two lines of code from the places that appear on the search results.